Interoperability Between Homogeneous and Heterogeneous Blockchains

Adam Svitek
Faculty of Informatics and Information Technologies
Slovak University of Technology in Bratislava
Bratislava, Slovakia
xsviteka@stuba.sk

***Abstract*—Blockchain interoperability is essential for asset transfers across different networks, layers, and ecosystems. Existing solutions often support individual transfer paths, but multi-step transfers still require users to manually coordinate multiple protocols and interfaces. This paper presents a client-side application layer that combines Across Protocol and Snowbridge into one guided transfer workflow between Ethereum-based networks and the Polkadot ecosystem. The system creates an execution plan from user input and processes the required protocol steps in sequence. Testnet evaluation shows that the proposed solution primarily simplifies transfer coordination, while protocol-level performance remains dependent on the underlying interoperability mechanisms.**

***Index Terms*—Blockchain interoperability, cross-chain transfers, cross-layer transfers, Ethereum, Polkadot, Layer 2, Across Protocol, Snowbridge**

I. Introduction

Blockchain interoperability has become an important requirement for modern decentralized systems. Blockchain use is no longer limited to isolated networks. It now includes multiple execution layers, heterogeneous blockchain architectures, and different asset transfer mechanisms. This is especially visible in the Ethereum ecosystem, which extends beyond the main Layer 1 network to a growing set of Layer 2 networks designed to improve scalability and reduce transaction costs. As users and decentralized applications increasingly operate across several networks, the ability to move assets between layers and ecosystems becomes more important.

Users may need to transfer assets between Ethereum Layer 2 networks, move assets from Layer 2 to Ethereum Layer 1, or continue further to an external ecosystem such as Polkadot. Although many interoperability protocols and bridges already support parts of these transfer paths, they often focus on one specific part of the route. These mechanisms differ in their interfaces, supported networks, transaction flow, and execution logic. As a result, multi-step transfers may still require the user to manually combine several tools and execute the required steps in the correct order.

This paper addresses the problem of fragmented transfer workflows by proposing a unified application layer for selected transfers between Ethereum-based networks and the Polkadot ecosystem. The proposed solution combines Across Protocol for transfers inside the Ethereum ecosystem with Snowbridge for transfers between Ethereum and Polkadot. Instead of requiring the user to manually select and coordinate multiple interoperability tools, the application prepares a transfer plan and executes the required protocol steps in sequence.

The main contribution of the paper is a client-side application prototype that presents selected single-step and multi-step transfer scenarios through one guided workflow. The goal is not to replace the underlying interoperability protocols, but to reduce user-facing complexity by coordinating them at the application layer.

II. Background

A. Blockchain and interoperability

Blockchain is a type of distributed ledger technology that enables a network of participants to maintain a shared and synchronized database without relying on a central authority [1], [2]. Transactions are grouped into blocks, which are cryptographically linked to form a chain [2]. This structure improves transparency and resistance against unauthorized modification, because changing stored data would require changing subsequent blocks and gaining agreement from the network [1].

The first widely adopted blockchain use case was introduced by Nakamoto in Bitcoin as a peer-to-peer electronic cash system [1]. Bitcoin used Proof of Work as part of its decentralized consensus mechanism, building on earlier proof-of-work ideas such as Hashcash [1], [3], [4]. Consensus mechanisms such as Proof of Work and Proof of Stake allow decentralized networks to agree on a valid state without centralized control [5], [6]. While Proof of Work relies on computational effort and mining, Proof of Stake is based on economic commitment of validators [7], [8]. These mechanisms provide security and consistency, but also introduce trade-offs such as energy consumption, scalability limits, and protocol complexity [9]–[11].

Over time, blockchain systems evolved beyond simple payment transfers. Ethereum extended blockchain usage by introducing smart contracts, which are self-executing programs stored and executed on the blockchain [12]–[14]. This enabled decentralized applications and more complex asset interactions. Ethereum operates as a Layer 1 blockchain providing

consensus, security, and data availability. To improve scalability and reduce transaction costs, Layer 2 networks were introduced. These networks execute transactions outside the main Ethereum network while still relying on Ethereum Layer 1 for security.

Blockchain interoperability refers to the ability of independent blockchain networks to exchange data and assets while maintaining security and correctness [15], [16]. Since blockchain ecosystems evolve independently and use different architectures, execution environments, and trust models, enabling communication between them is not a trivial problem [16], [17]. Existing interoperability solutions include notary schemes, relays, sidechains, and hash-locking mechanisms, each introducing different trade-offs in terms of security, trust assumptions, and performance [18].

B. Cross-chain and cross-layer transfers

In this work, it is useful to distinguish between cross-chain and cross-layer transfers. Cross-chain communication refers to interaction between independent blockchain ecosystems, for example Ethereum and Polkadot. These systems do not share the same consensus mechanism, state, or execution environment [19], [20]. As a result, cross-chain transfers usually require mechanisms that can coordinate or verify actions between heterogeneous blockchain environments.

Cross-layer transfers, on the other hand, happen inside one broader ecosystem, for example between Ethereum Layer 1 and Ethereum Layer 2 networks. Layer 2 networks are introduced to improve scalability and reduce transaction costs, while still relying on Ethereum Layer 1 for security. Although this makes them more closely related to Ethereum than fully independent blockchains, asset movement between layers still introduces practical complexity for users [21].

This distinction is important for the proposed solution. Transfers inside the Ethereum ecosystem can be treated as cross-layer or EVM-to-EVM transfers, while transfers between Ethereum and Polkadot are cross-chain transfers between heterogeneous ecosystems. Many interoperability solutions focus only on one of these areas. Some protocols are optimized for fast transfers inside the Ethereum ecosystem, while others focus on secure communication between different blockchain ecosystems.

Despite significant progress in interoperability research and development, many solutions remain limited to specific network pairs, ecosystems, or use cases [17], [18]. As a result, users may need to interact with multiple systems and interfaces when executing more complex transfer routes. This increases operational complexity and introduces additional risks [16], [18].

C. Integrated protocols

The proposed solution uses two existing interoperability protocols: Snowbridge and Across Protocol. These protocols were selected because they support different parts of the overall transfer path.

Snowbridge is an interoperability protocol designed for communication between the Ethereum and Polkadot ecosystems while minimizing trust assumptions [22]. Snowbridge uses light clients, which allow one blockchain ecosystem to verify information from another without relying only on trusted external validators. In this model, cross-chain messages are validated using cryptographic verification instead of a centralized middle party. This improves security compared to bridges that depend mainly on centralized validators.

Snowbridge enables the transfer of assets and messages between Ethereum and Polkadot parachains, including Asset Hub [23]. Assets originating on Ethereum can be transferred to the Polkadot ecosystem, and assets or messages from the Polkadot side can also be transferred back to Ethereum. This approach provides stronger trust minimization, but it also introduces higher complexity and longer execution times compared to simpler liquidity-based transfer mechanisms.

Across Protocol is an interoperability solution designed for efficient asset transfers inside the Ethereum ecosystem, mainly between Ethereum Layer 2 networks and Ethereum Layer 1 [24]. Across uses an intent-based architecture combined with a relayer-driven liquidity model. The user defines the transfer intent, including the origin chain, destination chain, asset, recipient, and amount. After the user deposits assets on the origin chain, relayers provide liquidity on the destination chain by sending funds to the selected recipient [25].

This design allows users to receive assets faster than when waiting for some native cross-chain finality mechanisms. Across also uses an *optimistic security model*, where transfers are assumed valid but can be challenged during a defined period. The key idea is the separation between fast user execution and delayed relayer settlement. This makes Across suitable for the EVM-to-EVM and cross-layer part of the proposed solution.

Together, Snowbridge and Across Protocol cover two different interoperability needs. Snowbridge provides Ethereum-to-Polkadot interoperability, while Across provides faster transfers inside the Ethereum ecosystem. The proposed application combines these two protocols at the application layer, allowing selected transfer scenarios to be presented as one guided workflow instead of separate manual steps.

III. Related Works

Existing interoperability solutions aim to connect fragmented blockchain ecosystems by enabling asset transfers and messaging across different networks. These solutions differ in their architecture, trust assumptions, and performance. While some protocols aim to connect layers within one system, others try to achieve better cross chain interoperability.

Interoperability protocols can be divided into two main groups, those that want to achieve faster transfer speed by introducing liquidity pools or relayers where the main downside is additional trust assumptions or delayed settlement with relayers and others focusing mostly on minimizing trust to reduce reliance on central entity by using cryptographic ver-

ification mechanisms such as light-clients or zero-knowledge proofs.

While these solutions improve usability and performance in specific scenarios, many of them are still limited to selected ecosystems, network pairs, or compatible blockchain environments. This creates space for solutions that combine existing interoperability mechanisms into a more unified transfer workflow.

A. HyperBridge

Hyperbridge is an interoperability protocol designed to enable secure communication between heterogeneous blockchain systems using cryptographic verification. Hyperbridge is based on verifiable state and consensus proofs, allowing one chain to verify the state of the other independently.

The main advantage of Hyperbridge is its strong trust-minimized security model. By relying on cryptographic proofs instead of external validators or liquidity providers, it significantly reduces trust assumptions and improves the overall integrity of cross-chain communication. Its design also allows interoperability between different blockchain ecosystems instead of focusing only on transfers inside one compatible environment.

However, this design comes with increased complexity because of the generation and verification of cryptographic proofs. Higher complexity may also lead to higher computational cost, increased transaction fees, and higher latency [26].

B. TurtleV2

TurtleV2 was an interoperability platform developed by Velocity Labs that aimed to simplify cross-chain asset transfers within the Polkadot ecosystem. TurtleV2 did not introduce a new interoperability mechanism, but instead integrated multiple existing protocols into a unified workflow. Its goal was to reduce the complexity of multi-step cross-chain operations and combine them into a more user-friendly process.

The platform focused on scenarios involving transfers across multiple layers and ecosystems, such as bridging assets from Ethereum to Polkadot and then routing them to specific parachains. To achieve this, TurtleV2 combined Snowbridge for Ethereum-Polkadot transfers and XCM for internal Polkadot transfers. This enabled a workflow from Ethereum to Polkadot and its parachains within one user interface.

Since TurtleV2 relied on existing protocols, it did not introduce a new security model for cross-chain transfers. Instead, its security and trust assumptions depended on the protocols it integrated [27].

C. Hop Protocol

Hop Protocol is an interoperability solution designed to provide fast and efficient transfers between Ethereum Layer 1 and Layer 2 networks, as well as between different Layer 2 networks. Its main goal is to reduce delays connected with rollup withdrawals and improve the efficiency of cross-layer transfers inside the Ethereum ecosystem. The protocol allows users to move assets across supported networks with lower latency compared to some native bridging mechanisms, improving overall usability. [28]

The main advantage of Hop Protocol is its ability to provide near-instant transfers across Layer 2 networks. This creates a smoother experience for users who need to transfer assets frequently between rollups without waiting for longer settlement periods. The main limitation is the need for sufficient liquidity to function efficiently [28].

D. cBridge

Celer cBridge is an interoperability solution designed to enable faster asset transfers across multiple blockchain networks, including Ethereum Layer 1 and Layer 2 ecosystems. Similar to other speed-oriented transfer protocols, cBridge allows users to transfer assets without waiting for slow native finality mechanisms. On the other hand, cBridge also relies on liquidity providers and validators, which introduces additional trust assumptions compared to fully trust-minimized solutions [29].

IV. Proposed Solution

A. Problem Addressed

Nowadays, many interoperability protocols support specific transfer paths for asset transfers across Ethereum Layer 2 networks, Ethereum Layer 1, and external ecosystems such as Polkadot. However, these transfer paths are still often fragmented and multi-step transfers cannot be completed easily. A user may be able to complete each part of the transfer with an existing protocol, but the whole process is usually not offered as one continuous operation [19].

This problem becomes more visible in multi-step transfers. In such cases, the user often has to work with more than one protocol and therefore more than one interface. For example, a transfer from an Ethereum Layer 2 network to the Polkadot ecosystem may require first moving assets to Ethereum Layer 1 and then continuing to the target ecosystem.

As a result, the transfer process becomes harder to use and the user is prone to making more mistakes. The user must decide which protocol should be used for each part of the route, switch between different applications, and execute the steps in the correct order. This increases the risk of mistakes, especially for users who are not familiar with the process of asset transfers.

From this perspective, the main problem is not the lack of interoperability protocols, but the lack of a unified workflow that would connect them into one user-oriented transfer process. Many existing solutions solve only one part of the problem, while the full end-to-end transfer is still left for the user to coordinate.

B. Solution Concept

To address this problem, the proposed solution introduces an application layer that unifies the workflow for asset transfers between Ethereum-based networks and the Polkadot ecosystem. The main goal was not to create a new interoperability protocol, but to combine existing ones into one larger single

transfer process that is easier for the user to understand and use.

The proposed solution is based on two different existing protocols with two different roles in the final implementation. Across Protocol is used for transfers within the Ethereum ecosystem, mostly between Layer 2 networks and Ethereum Layer 1. The second one is Snowbridge, which is used for transfers between Ethereum and Polkadot [22], [24]. By combining these two protocols into one application, the system can support both direct transfers and more complex multi-step scenarios.

The main idea behind this solution is to separate user input from the actual execution logic and reduce the need for the user to understand the internal operation of the protocols. The user provides only the essential transfer parameters, such as the source network, destination network, selected asset, amount, and recipient address. Based on this information, the application determines the required transfer path and prepares the sequence of steps needed for transfer execution.

This approach allows the system to hide much of the complexity that would otherwise remain on the user. Instead of visiting and working with multiple platforms and intermediate steps, the user interacts with one unified interface that represents the transfer as one process. In this way, the proposed solution acts as a coordination layer above existing interoperability mechanisms, while keeping the original execution model of each integrated protocol.

C. Supported Transfer Scenarios

The solution in its current state is designed to support several transfer scenarios that combine interoperability mechanisms across Ethereum networks and the Polkadot ecosystem. These scenarios differ in the number of steps required for successful completion and in the protocol used during transfer execution.

The first supported scenario consists of transfers between Ethereum-based networks, including Ethereum Layer 1 and supported Ethereum Layer 2 networks. In this case, the transfer is handled as a single-step process using only Across Protocol. This scenario is used for direct asset movement inside the Ethereum ecosystem [24].

The second scenario consists of transfers from Ethereum Layer 1 to the Polkadot ecosystem and in the opposite direction. In this case, the transfer is executed through Snowbridge, which provides interoperability between Ethereum and Polkadot. From the user's perspective, the use of a different protocol is visible mainly in the transfer preview and required wallet interaction, but the general workflow of the application remains the same. This removes the need for the user to manually switch between separate tools.

The last and most important scenario consists of transfers from a supported Ethereum Layer 2 network to the Polkadot ecosystem. In this case, the transfer cannot be completed using only one protocol, and the cooperation of both protocols is required. The system combines two dependent steps that follow each other according to the selected transfer direction.

D. End-to-End Transfer Workflow

The end-to-end transfer workflow begins with the user entering the necessary information for the transfer. The interface is designed so that the user provides transfer parameters but does not need to manually select the interoperability protocol suitable for the specific transfer. This decision is handled by the application logic.

After the source network is selected, the application loads the assets that can be transferred from the chosen source. Once the asset is selected, the application determines the destinations that are currently supported for the selected combination of network and asset. This dynamic selection is important because not every asset can be transferred through every path, and availability is limited by the integrated protocols. This also helps prevent invalid combinations during input selection.

The last required inputs are the amount and the recipient address. Since different ecosystems require different address formats, the application also validates the correctness of the address to reduce the submission of invalid transfer requests. Once all required parameters are provided, the application creates a transfer preview.

The preview represents the transfer as a structured plan that shows the individual steps needed for execution. Depending on the selected route, the preview may contain only one step, such as a direct Across transfer or a direct Snowbridge transfer, or it may contain multiple consecutive steps in the case of a combined route. This preview gives the user a chance to review the planned operation before any transaction is submitted.

After the user confirms the transfer, the application begins the execution phase. The transfer plan is processed step by step in the order defined by the planner. For each step, the application prepares the required transaction data, switches to the correct network if needed, and sends the transaction request to the connected wallet for user approval. Once the transaction is approved and submitted, the system continues with the corresponding execution logic.

In single-step transfers, this process is relatively direct, since the transfer is completed within one protocol interaction. In multi-step transfers, however, the workflow depends on coordination between intermediate results. For example, in a transfer from an Ethereum Layer 2 network to the Polkadot ecosystem, the first step moves the asset from Layer 2 to Ethereum Layer 1, while the second step continues from Ethereum to the final destination in Polkadot.

For such combined transfers, the system must wait until the previous step is completed before the next one can begin, since the following step depends on the asset becoming available in the intermediate environment. This sequential coordination prevents the transfer from failing due to missing funds on the intermediate chain and allows the workflow to operate as one dependent process rather than as separate isolated actions.

During execution, the application also provides continuous feedback about the progress of the transfer, including the currently active step, completed steps, and possible errors. After all planned steps are completed successfully, the final result is presented to the user. From the user's perspective,

the whole transfer is therefore represented as one continuous workflow, even though the underlying execution may involve multiple protocols, networks, and transaction stages.

V. System Architecture

The proposed solution is implemented as a client-side web application built with React, TypeScript, and Vite. The application communicates directly with external interoperability services, blockchain RPC endpoints, and the connected wallet. This design removes the need for a dedicated backend component and keeps the system lightweight, while the user remains in direct control of transaction approval through their own wallet. Instead of introducing a separate execution authority, the application acts as an orchestration layer that connects protocol data, user input, and transaction execution within one environment.

Figure 1 shows the high-level architecture of the proposed solution. The user interacts with the client-side web application, which validates input, discovers available routes and assets, creates a transfer plan, and executes the required protocol steps. External systems include the user wallet, RPC providers, Across Protocol, and Snowbridge. The actual transfers are executed on Ethereum Layer 2 networks, Ethereum Layer 1, and Polkadot Asset Hubs.

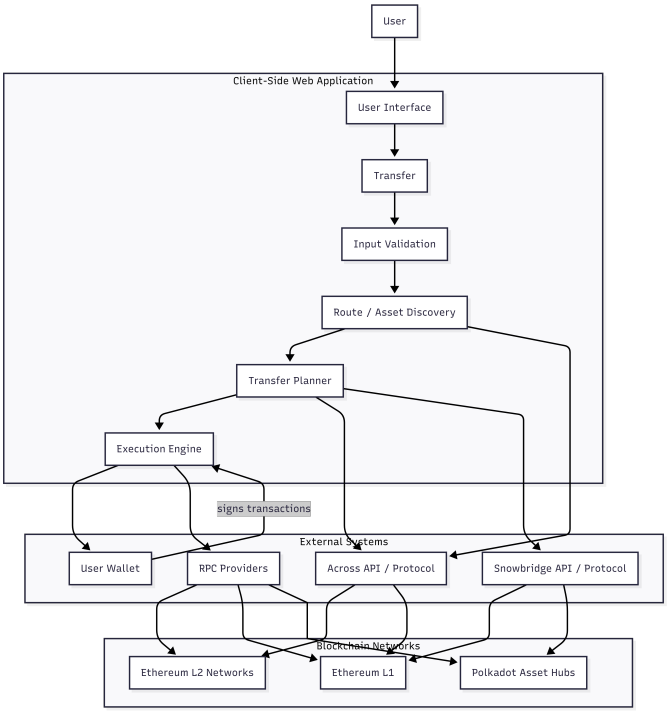

Fig. 1. High-level architecture overview of the proposed application.

From a logical point of view, the architecture is composed of several cooperating parts. The user interface is responsible for collecting transfer parameters, displaying the generated transfer preview, and presenting execution progress. A data and catalog layer maintains the set of supported networks, assets, and routes. For Ethereum-based transfers, route data are loaded dynamically from the Across API [30], while supported Polkadot destinations are integrated into the same internal chain model. This allows the application to work with one unified representation of transfer targets, even though the underlying ecosystems use different addressing and execution models.

Another central part of the architecture is the transfer planner, which converts the transfer request into an executable plan. Based on the selected source, destination, asset, and environment, the planner determines whether the transfer can be completed in a single step or whether it requires a combined route with multiple dependent steps. The execution layer then processes this plan and interacts with the required protocol interfaces. Its role is to prepare transaction requests, invoke the corresponding protocol actions, and preserve the correct execution order when later actions depend on the successful completion of earlier ones.

Internally, the application uses a structured transfer representation that separates user-level input from protocol-specific execution logic. This makes it possible to describe both direct transfers and more complex multi-step routes through one consistent internal model. As a result, the architecture can coordinate heterogeneous transfer scenarios without exposing their full technical complexity directly to the user.

VI. Implementation

The implementation addresses the fragmentation of multi-step asset transfers by introducing an internal execution plan between the user interface and the interoperability protocols. Instead of requiring the user to select the correct bridge and manually coordinate the order of operations, the application transforms a transfer request into one or more executable steps. This implementation allows different transfer scenarios to be handled through one common structure.

One of the main implementation goals is to prevent invalid transfer configurations before any transaction is submitted. Supported assets, destination networks, and transfer paths are derived from available routes provided by the protocol APIs rather than being treated as independent user inputs. After the user selects the initial parameters, the interface filters out unsupported combinations and only allows routes that can be processed by the system. Address validation is also performed before execution, because Ethereum-based networks and the Polkadot ecosystem use different address formats.

The planner is responsible for transforming user input into the required protocol steps. Transfers within Ethereum-based networks are represented through Across Protocol, while transfers between Ethereum and Polkadot are handled through Snowbridge. In combined routes, such as transfers from an Ethereum Layer 2 network to the Polkadot ecosystem, the first step must be completed before the following step can be executed. The execution layer then processes the generated plan in sequence, prepares the required transaction data, requests wallet approval, submits the protocol actions, and updates the execution state.

This step-based structure also improves the extensibility of the solution. Since different routes are represented as ordered execution steps, additional bridges, SDKs, or routing mechanisms could be integrated as new step types without redesigning the whole application. The general workflow, execution ordering, wallet interaction, and progress tracking can remain the same, while the protocol-specific transaction preparation and route integration would be implemented separately.

The sequence diagram in Fig. 2 illustrates this approach for the most complex supported scenario, where one user request is executed through two dependent protocol actions.

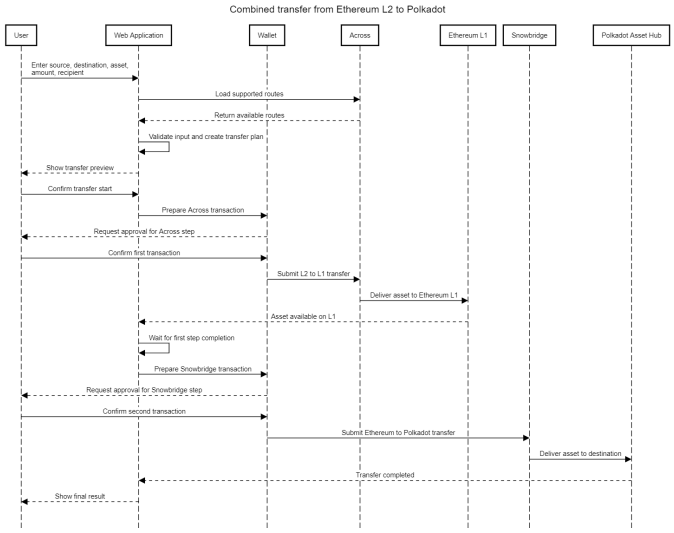

Fig. 2. L2 to Polkadot transfer sequence diagram

VII. Evaluation and Discussion

Since the proposed solution does not replace any interoperability protocol, protocol-level properties such as bridge security, liquidity availability, and transaction fees remain dependent on the protocols themselves. Therefore, the evaluation focuses on the application layer and on the practical behavior of the implemented workflow during testnet executions.

From the workflow perspective, the proposed application reduces the number of separate interfaces that the user has to work with when more than one protocol is required. It also removes the need for the user to manually search for the correct route across different chains and protocols. Instead, the application determines supported transfer paths based on the selected source chain, asset, and destination chain.

A. Workflow-level comparison

The evaluation does not include a direct measurement of the time required by users to complete the same transfer manually across several tools. Such a measurement would require a separate user study with multiple users, controlled conditions, and comparable levels of technical experience. Therefore, the workflow-level comparison focuses on observable properties of the process, such as the number of required interfaces, manual protocol selection, route planning, step ordering, and progress tracking.

TABLE I
Workflow comparison between manual and proposed transfer execution

Criterion	Manual	Proposed
Interfaces	Multiple tools	One interface
Protocol choice	Manual	Automatic
Route planning	Manual	Generated
Step ordering	Manual	Sequential
Progress tracking	Split	Unified
Address validation	Tool-dependent	Application-level

As shown in Table I, the proposed application mainly reduces the coordination effort required from the user. The user still has to approve transactions through the connected wallet, but the application prepares the route, shows the transfer plan, and executes the required steps in the correct order.

B. Testnet execution measurements

To evaluate the practical behavior of the implementation, repeated transfers were performed in testnet environments. The tests were conducted on the same machine, an Asus Zenbook 14X, in a Google Chrome web browser with the application running locally.

The measured scenarios included transfers between Ethereum Layer 2 test networks, transfers from an Ethereum Layer 2 test network to Ethereum Layer 1, transfers from Ethereum Layer 1 to Westend Asset Hub, and transfers from an Ethereum Layer 2 network to Westend Asset Hub.

The measured metric was the transfer execution time displayed by the testing version of the application after successful completion. The collected times were then used to calculate averages and ranges for each tested scenario. This provided a clearer overview of the observed transfer behavior and made it possible to compare the supported scenarios.

User wallet confirmation time was not included in the measured metric, because the confirmation time depends on the response time of the user and not only on the transfer workflow or the underlying protocols.

TABLE II
Observed execution times in testnet transfers

Scenario	Avg. time	Range
L2 to L2	25.59 s	24.95–27.07 s
L2 to L1	25.16 s	24.38–25.75 s
L1 to Westend	20 min 50 s	20 min 14 s–21 min 26 s
L2 to Westend	23 min 6 s	22 min 6 s–24 min 32 s

As shown in Table II, transfers inside the Ethereum ecosystem were completed in relatively short time intervals and averaged around 25 seconds. Transfers involving Westend Asset Hub required significantly more time and averaged approximately 21 to 23 minutes.

C. Discussion

The evaluation results show two different aspects of the proposed solution. The workflow-level comparison shows how

the application reduces manual coordination effort, while the testnet measurements show the practical execution behavior of the supported transfer scenarios.

The results show a clear difference between token transfers performed inside the Ethereum ecosystem and transfers involving an external blockchain ecosystem. Transfers from Layer 2 to Layer 2 and from Layer 2 to Layer 1 were completed quickly, with average execution times of approximately 25 seconds. In contrast, transfers between Ethereum and Westend Asset Hub required significantly longer completion times, exceeding 20 minutes on average. These results show higher latency in cross-ecosystem transfers compared to transfers executed within the Ethereum ecosystem.

The longest observed average time was measured for transfers from an Ethereum Layer 2 network to Westend Asset Hub. However, this time was not significantly different from transfers from Ethereum Layer 1 to Westend Asset Hub. This indicates that most of the execution time was caused by the Snowbridge part of the workflow, while the additional Across step from Layer 2 to Layer 1 added only a relatively small part of the total transfer time.

These findings indicate that the proposed application does not primarily improve protocol-level transfer speed. Its main contribution is the simplification and coordination of multiple transfer steps into one workflow. The application can reduce manual coordination effort, because the user does not need to manually identify the correct protocols, applications, and execution order for supported transfer paths. The actual time saved by users was not measured directly, because that would require a separate user study.

The presented measurements should not be interpreted as a universal performance benchmark. They were collected in testnet environments and are highly dependent on network conditions, relayer behavior, RPC availability, bridge processing, and the availability of supported routes.

VIII. Conclusion

This paper addressed the problem of fragmented asset transfers across Ethereum-based networks and the Polkadot ecosystem. Although existing interoperability protocols already provide mechanisms for cross-layer and cross-chain transfers, users are often required to manually combine multiple tools, interfaces, and transaction steps. This increases the complexity of the process and creates space for user errors.

The proposed solution introduced a client-side application layer that combines Across Protocol and Snowbridge into one guided transfer workflow. The implementation allows the user to define the source network, destination network, asset, amount, and recipient address, while the application prepares the required transfer plan and executes the necessary protocol steps in the correct order. In this way, the system does not replace the interoperability protocols, but coordinates them through a unified user-facing process.

The evaluation showed that Ethereum-based transfers were completed significantly faster than transfers involving the Polkadot ecosystem. However, the main contribution of the proposed solution is not protocol-level performance improvement, but the simplification of multi-step transfer coordination. The application reduces the need for manual routing decisions and separates user input from protocol-specific execution details.

Future work may focus on extending the number of supported protocols and improving route selection based on fees and execution time. Additional monitoring of transaction states could also improve transparency during longer cross-ecosystem transfers. Since the proposed solution is designed as an application-level coordination layer, it could be extended with additional protocols or SDKs to support more transfer routes and blockchain networks. The long-term goal would remain the same: reducing decision complexity and making cross-chain transfers easier for the end user.

References

[1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," https://bitcoin.org/bitcoin.pdf, Oct. 2008, accessed: 2026-04-30.

[2] A. S. Rajasekaran, M. Azees, and F. Al-Turjman, "A comprehensive survey on blockchain technology," *Sustainable Energy Technologies and Assessments*, vol. 52, p. 102039, 2022.

[3] C. Dwork and M. Naor, "Pricing via processing or combatting junk mail," in *Advances in Cryptology – CRYPTO '92*. Springer, 1993, pp. 139–147.

[4] A. Back, "Hashcash: A denial of service counter-measure," https://www.hashcash.org/hashcash.pdf, 2002, accessed: 2026-04-30.

[5] J. Xu, C. Wang, and X. Jia, "A survey of blockchain consensus protocols," *ACM Computing Surveys*, vol. 55, no. 13s, pp. 1–35, 2023.

[6] D. P. Oyinloye, J. S. Teh, N. Jamil, and M. Alawida, "Blockchain consensus: An overview of alternative protocols," *Symmetry*, vol. 13, no. 8, p. 1363, 2021.

[7] S. King and S. Nadal, "Ppcoin: Peer-to-peer crypto-currency with proof-of-stake," https://decred.org/research/king2012.pdf, 2012, accessed: 2026-04-30.

[8] C. T. Nguyen, D. T. Hoang, D. N. Nguyen, D. Niyato, H. T. Nguyen, and E. Dutkiewicz, "Proof-of-stake consensus mechanisms for future blockchain networks: Fundamentals, applications and opportunities," *IEEE Access*, vol. 7, pp. 85 727–85 745, 2019.

[9] J. Sedlmeir, H. U. Buhl, G. Fridgen, and R. Keller, "The energy consumption of blockchain technology: Beyond myth," *Business & Information Systems Engineering*, vol. 62, pp. 599–608, 2020.

[10] L. W. Cong, Z. He, and J. Li, "Decentralized mining in centralized pools," *The Review of Financial Studies*, vol. 34, no. 3, pp. 1191–1235, 2021.

[11] V. Buterin and V. Griffith, "Casper the friendly finality gadget," 2017.

[12] V. Buterin, "Ethereum: A next-generation smart contract and decentralized application platform," https://ethereum.org/whitepaper/, 2014, accessed: 2026-04-30.

[13] Ethereum Foundation, "What is ethereum?" https://ethereum.org/what-is-ethereum/, 2026, accessed: 2026-04-30.

[14] N. Szabo, "Formalizing and securing relationships on public networks," *First Monday*, vol. 2, no. 9, 1997.

[15] K. Ren, N.-M. Ho, D. Loghin, T.-T. Nguyen, B. C. Ooi, Q.-T. Ta, and F. Zhu, "Interoperability in blockchain: A survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 12, pp. 12 750–12 769, 2023.

[16] S. D. Kotey, E. T. Tchao, A.-R. Ahmed, A. S. Agbemenu, H. Nunoo-Mensah, A. Sikora, D. Welte, and E. Keelson, "Blockchain interoperability: The state of heterogenous blockchain-to-blockchain communication," *IET Communications*, vol. 17, no. 8, pp. 891–914, 2023.

[17] D. Mohanty, D. Anand, H. M. Aljahdali, and S. G. Villar, "Blockchain interoperability: Towards a sustainable payment system," *Sustainability*, vol. 14, no. 2, p. 913, 2022.

[18] G. Wang, Q. Wang, and S. Chen, "Exploring blockchains interoperability: A systematic survey," *ACM Computing Surveys*, vol. 55, no. 13s, pp. 1–38, 2023.

[19] R. Belchior, A. Vasconcelos, S. Guerreiro, and M. Correia, "A survey on blockchain interoperability: Past, present, and future trends," *ACM Computing Surveys*, vol. 54, no. 8, pp. 1–41, 2021.
[20] P. Han, Z. Yan, W. Ding, S. Fei, and Z. Wan, "A survey on cross-chain technologies," *Distributed Ledger Technologies: Research and Practice*, vol. 2, no. 2, pp. 1–30, 2023.
[21] C. Sguanci, R. Spatafora, and A. M. Vergani, "Layer 2 blockchain scaling: A survey," 2021.
[22] Snowbridge, "Snowbridge documentation," https://docs.snowbridge.network/, 2026, accessed: 2026-04-30.
[23] ——, "Architecture overview," https://docs.snowbridge.network/architecture/overview, 2026, accessed: 2026-04-30.
[24] Across Protocol, "Across developer documentation," https://docs.across.to/, 2026, accessed: 2026-04-30.
[25] ——, "Actors in the system," https://docs.across.to/reference/actors-in-the-system, 2026, accessed: 2026-04-30.
[26] Hyperbridge, "Hyperbridge documentation," https://docs.hyperbridge.network/, 2026, accessed: 2026-04-30.
[27] Velocity Labs, "Turtle v2: Evolving cross-chain token transfers in the polkadot ecosystem," https://medium.com/@velocity-labs/turtle-v2-evolving-cross-chain-token-transfers-in-the-polkadot-ecosystem-09d15ff1e6a4, Dec. 2024, accessed: 2026-04-30.
[28] C. Whinfrey, "Hop: Send tokens across rollups," https://hop.exchange/whitepaper.pdf, Jan. 2021, accessed: 2026-04-30.
[29] Celer Network, "Celer cbridge documentation," https://cbridge-docs.celer.network/, 2023, accessed: 2026-04-30.
[30] Across Protocol, "Api reference," https://docs.across.to/api-reference, 2026, accessed: 2026-04-30.